
Manual técnico y de despliegue — Evento Cultural

Manual técnico/despliegue (artefacto 8 del marco ágil, anexo
metodológico)

Roberto Rizzo (frontend) · José Miguel Moncada (BD/QA) · David Herrera (backend
e integraciones)

25 de septiembre de 2026

Proyecto Capstone APT122 · Duoc UC · Sede Maipú

1. Composición del sistema

Parte	Tecnología	Rol
Scraper	Python (conectores, NVIDIA NIM, checkpoints SQLite)	Recolección, normalización y enriquecimiento de eventos desde 21 fuentes públicas
Base de datos	Supabase (PostgreSQL + RLS)	Catálogo canónico, bitácora de ejecuciones y evidencia de extracción
Frontend	Angular 22 + Ionic 9 (web y móvil)	Descubrimiento: búsqueda, filtros, mapa, calendario y recomendaciones
Automatización	GitHub Actions	Ejecución periódica/manual del scraper con pruebas

Cuadro 1: Componentes del sistema.

2. Ejecución local

2.1 Scraper (Windows)

```
instalar.bat      REM crea venv, instala dependencias y Chromium de Playwright
ejecutar.bat      REM pipeline completo (extraccion + exportacion + sincronizacion opcional)
python -m unittest discover -s tests -v
```

Las variables de entorno se documentan en `.env.example` del proyecto; incluyen credenciales de Supabase y el endpoint de NVIDIA NIM. **No** versionar secretos: el workflow los consume como secretos de GitHub.

2.2 Frontend

```
npm install
npm start          # ng serve en http://localhost:8100
ng test            # pruebas (Vitest)
npx ng build --configuration production
```

En Windows existen además `install.bat` y `run.bat` para el entorno local. El frontend funciona con catálogo mock en memoria; la conexión a Supabase es un pendiente declarado.

3. Estrategia de despliegue (sin Docker)

El equipo decidió **no usar Docker**; el anexo metodológico se cumple documentando esta decisión y el estado real. El despliegue se compone así:

El sistema no se aloja en contenedores: las plataformas de alojamiento y automatización son GitHub Actions, Vercel y Capacitor. El estado de cada componente:

3.1 Scraper en GitHub Actions (desplegado, en testeo)

- El workflow está desplegado y el equipo se encuentra probando el despliegue: correr el pipeline con pruebas y sincronización a Supabase, usando secretos de repositorio (nunca credenciales en el código).
- Evidencia de ejecución: captura `Evidencia_ejecución_scraper_en_GitHub_Actions.PNG` en Evidencias Proyecto.

Componente	Plataforma	Estado
Scraper	GitHub Actions (workflow <code>scrape-events.yml</code>)	Desplegado, con ejecución manual y cron diario activo desde el 24-09 (00:37, America/Santiago; corridas #9-#11 verificadas del 25 al 27-09, evidencia en Evidencias Proyecto); la corrida del 27-09 falló en pruebas (Plan de pruebas §5.1)
Web	Vercel	La página se alojará en Vercel: despliegue en preparación
Móvil (Android)	Capacitor 8 + Android Studio	La misma web se convertirá en aplicación mediante Capacitor 8: conversión en preparación (dependencias presentes en <code>package.json</code>)

Cuadro 2: Estrategia de despliegue por componente y su estado real.

- La periodicidad ya está definida y activa: **schedule** con cron diario a las 00:37 (America/Santiago) desde el 24-09. Para dar el despliegue por completado, el equipo debe corregir la prueba dependiente del tiempo del 27-09 (Plan de pruebas §5.1) y obtener una corrida programada completa en verde.

3.2 Web en Vercel (alojamiento en preparación)

1. Conectar el repositorio del frontend a Vercel (framework Angular/Ionic detectado automáticamente o con `ng build -configuration production` como comando de build).
2. Configurar variables de entorno de Supabase en el proyecto de Vercel (mismas claves que `.env.example`).
3. Publicar y registrar la URL como evidencia de despliegue.

3.3 De la web a la app: Capacitor (conversión en preparación)

La aplicación Android es la misma web envuelta con Capacitor; pasos de la conversión:

1. `npx cap add android` (si la plataforma no existe) y `npx cap sync`.
2. Abrir en Android Studio, compilar el APK/AAB de prueba y registrar la evidencia de la compilación.

4. Variables de entorno

Grupo	Uso
Supabase (URL y clave de servicio)	Sincronización del scraper y, en el futuro, del frontend
NVIDIA NIM (endpoint y clave)	Enriquecimiento de eventos en el pipeline

Cuadro 3: Grupos de variables de entorno (nombres exactos en `.env.example`).

5. Solución de problemas comunes

- **Pipeline detenido a mitad:** los checkpoints en SQLite permiten reanudar; volver a ejecutar `ejecutar.bat`.

- **Consola del navegador con rechazos de View Transitions:** los rechazos benignos ya se suprimen en `main.ts`; cualquier otro error de consola se trata como falla de la verificación.
- **Puerto 8100 ocupado:** liberar el proceso de `ng serve` previo.